

Boundary Case: Deployment-Evolution Rule Version Compatibility as Standalone Architectural Treatment — Formalizing How CKS Maintains Coherence Across Long Deployment Lifecycles Where Multiple Rule Versions Coexist, Through A2.40 Field 4 Specific Rule Version Recording and Rule-Authored Compatibility per A2.04 Without Auto-Migration of Historical State

Author: Wenxin Li (Independent Researcher) **ORCID:** 0009-0004-8065-3235 **Date:** May 7, 2026 **License:** Creative Commons Attribution 4.0 International (CC BY 4.0)

* * *

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize the *deployment-evolution rule version compatibility boundary* as a standalone architectural treatment — the specification of how a CKS deployment maintains coherence over a long lifecycle during which orchestration rules have been revised many times and substrate state authored under multiple rule versions coexists in substrate.

Abstract

A CKS deployment that runs for an extended period accumulates orchestration-rule revisions over its lifetime. Each revision is authored by humans under A2.04 and is itself a governance event under A1.01. Substrate state established under any historical rule version remains in substrate per A1.07; cells on the present substrate may encounter content authored under multiple rule versions and must operate coherently across them. This boundary case extends note A6.02's rule-retroactivity treatment from the local effect of one revision to the cumulative effect of many revisions across the deployment lifecycle. The architectural treatment has six elements: (i) field 4 of the A2.40 provenance metadata records the specific rule version under which each substrate change was authored; (ii) rules under A2.04 specify how they read state authored under prior versions; (iii) rule revisions are substrate-resident, each version preserved; (iv) replay per A4.06 uses the historical rule version recorded in provenance, not the current version; (v) migration of historical state is explicit human-authored re-processing per A6.02 that creates new substrate state alongside the original; (vi) deprecation of an old rule is itself a governance event recorded as substrate content. The note distinguishes the boundary from individual rule retroactivity (A6.02) and schema evolution (A6.15, the closing Phase A6 note), enumerates the canonical violations the architecture rules out, and provides the operational test the deployment must satisfy.

1. Why this boundary needs to be formalized as standalone

A deployment running for several months may not exercise rule version compatibility hard. A deployment running for several years almost certainly will. Rules authored per A2.04 are revised over time as the coordination domain evolves, as humans exercising override authority change what cells should do, and as

conflicts surface that prior rules under-specified. Each revision produces a new rule version. The substrate at any given moment carries content authored under some current selection of those versions and content authored under any number of historical versions, and cells operating on the substrate encounter both.

The boundary deserves standalone formalization for three reasons. First, the failure mode is invisible in the immediate effect of any single revision: A6.02 covers one revision's local treatment, but the cumulative effect of many revisions yields systems that work after the third revision and break after the thirtieth. Second, the boundary is consequential for the deployments where CKS's traceability and reproducibility commitments matter most — long-running coordination substrates, audit-required AI deployments, regulated-industry instantiations whose historical record must remain interpretable across years. Third, the treatment propagates back into how rules are authored: designers under A2.04 are authoring components of an evolving substrate whose interaction with historical rule versions is itself part of what the architecture commits to.

The boundary connects to the source paper at §5 (path retraceability and conflict), §3.3 (architectural and temporal qualifiers), §3.1 (substrate-as-source-of-truth and human-governance), and §11.3 (substrate as source of truth). What §3.3 commits to as "at any time" extends to "at any time across the deployment lifecycle"; what §3.1 and §11.3 commit to as substrate-as-source-of-truth extends to long-lifecycle source-of-truth, in which historical state and the rules that produced it remain in the authoritative artifact rather than being shed as the deployment evolves. A6.14 follows A6.01–A6.13 and precedes A6.15, which closes Phase A6 with the schema evolution / substrate migration boundary.

2. The boundary case scenario and what makes it non-obvious

A deployment has been operational for an extended period. During that period orchestration rules have been revised multiple times — sometimes adding rules, sometimes refining them, sometimes retiring rules that no longer fit the coordination domain. Each revision was a governance event under A1.01: authored by humans, recorded as substrate content, applied to cells executing afterward. Substrate state established under each rule version remains in substrate per A1.07. The substrate at the present moment therefore carries content whose authoring rule versions span the full history of the deployment.

What makes the scenario non-obvious is the joint interaction of three commitments the architecture preserves separately but that must hold together under long-lifecycle conditions. A1.07 retraceability requires that historical state remain interpretable from substrate content alone — the path back to its antecedents must be reconstructable years after authoring. A4.06 reproducibility requires that replay of a historical execution produce the original outcome, which requires the rule under which the original execution ran, not the current rule. A1.10 determinism requires that cell behavior on a given substrate input under a given rule be reproducible — across rule versions, this means the (cell, rule version, substrate state) tuple must remain identifiable. The three do not interact problematically when only one rule version exists; they interact across a long lifecycle as a coherent treatment — or fail to, in which case the deployment exhibits one of the canonical violations §5 enumerates.

3. Which architectural commitments are stressed

A2.04 — rule authoring. Rules are not only operationally specific to the workflow at the moment of authoring; they are components of an evolving substrate. Designers consider, at authoring time, how the

current rule reads state authored under prior rules, whether the new version requires re-processing of historical state, how rule version transitions are recorded in provenance, and whether old rules remain referenced by historical provenance. Compatibility is architectural surface, not implementation detail.

A2.40 field 4 — specific rule version recording. The six-field provenance metadata records, in field 4, the specific rule version under which each substrate change was authored. Recording "the change was made under rule R" is insufficient when R has had many versions; the trace must record "rule R version 17," because the path back through historical content runs through specific versions.

A1.07 — path retraceability across long timeframes. Historical state remains present, accessible, and interpretable; it is not auto-pruned or auto-translated to a current format that loses the original.

A4.06 — reproducibility across rule versions. Replay uses the historical rule version active at execution time. Reproducibility across versions is the operational test that detects whether the architectural treatment is in place.

A1.10 — determinism across rule versions. Deterministic guarantees hold across the (cell, rule version, substrate state) tuple as the deployment evolves, not only within a single rule version.

A1.01 — human-governed for version transitions. Authoring a new rule version, deprecating an old one, and choosing whether to re-process historical state are decisions under preserved human authority — not delegable to vendor policy, runtime middleware, LLM judgment, or external compliance frameworks.

A6.02 — rule retroactivity, extended. A6.02 establishes the local treatment: historical content remains as authored, the new rule applies prospectively, and migration is explicit human-authored re-processing. A6.14 extends this from the local effect of one revision to the cumulative effect across deployment lifetime, and adds the design-layer obligation on A2.04 that compatibility be considered at authoring time.

4. The architectural treatment

The treatment has six coupled elements.

(i) Specific rule version is recorded as provenance. Every substrate change carries, in A2.40 field 4, the specific rule version under which it was authored — a version identifier sufficient to address the rule as it stood at the moment of authoring. Historical state is therefore addressable to its authoring rule version.

(ii) Rules are authored for compatibility under A2.04. Rules specify, as part of rule content, how they read state authored under prior versions. A current rule may read prior-version state directly (when the prior format is compatible) or delegate to versioned interpretation logic (a rule clause that says, in effect, "for state authored under version N, interpret it as follows"). Either treatment is admissible; what is not admissible is silence about the question. A rule that does not specify how it processes historical state is incomplete in the architectural sense.

(iii) Rule revisions are substrate-resident; each version is preserved. A revision is itself a write to substrate, authored by humans under A1.01 governance, recorded with its own provenance. The prior version is not deleted; both become substrate-resident, and the deployment's history of rule versions receives the same source-of-truth treatment as substrate content as a whole.

(iv) Replay uses the historical rule version active at execution time. When a historical execution is replayed for reproducibility verification per A4.06, the replay engine uses the rule version field 4 of provenance points to, not the current rule version. Reproducibility is well-defined across deployment evolution because the original rule remains substrate-resident and addressable through the provenance chain.

(v) Migration of historical state is explicit human-authored re-processing per A6.02. When operators decide that historical state should be re-processed under a current rule version, humans author a rule that specifies what historical state is re-processed and how. The re-processing creates new substrate state alongside the original; the original is not overwritten. The new state carries provenance pointing to the migration rule and to the original; the original retains its own provenance and remains in substrate. This is A6.02's discipline applied across the longer arc of the lifecycle.

(vi) Deprecation of old rules is itself a governance event. When an old rule version ceases to govern new operations it is not silently removed. Deprecation is a human-authored decision recorded as substrate content. The deprecated rule remains substrate-resident as long as historical provenance references it — which, by A1.07, means as long as the historical content that points to it remains in substrate. Deprecation says only that the rule does not govern future operations; it does not say the rule is no longer authoritative content in the substrate's record.

These six elements are coupled. Removing any of them yields one of the canonical violations §5 enumerates.

5. Anti-pattern treatments that would violate the architecture

Rule-version-mismatch-causes-substrate-corruption. A current rule encounters historical state, fails to interpret it, and corrupts substrate content — overwriting it, marking it invalid, or producing inconsistent downstream content. Breaks A1.07 and A4.06.

Auto-migration-of-historical-state-to-new-rule-format. Historical state is automatically transformed to current format when a revision is authored, without explicit re-processing per A6.02; the original is replaced. Breaks A1.07, A6.02, and A1.01 (a substrate-wide rewrite occurs outside the inspect-modify-override authority architecture).

Silent-deprecation-of-old-rules. An old rule is removed from substrate when it ceases to govern new operations, even though historical provenance references it. Breaks A4.06 (replay can no longer locate the original rule) and A1.07 (the path through historical content runs through a rule the substrate no longer carries).

Version-erasure. Provenance is recorded without the specific rule version (field 4 records "rule R" rather than "rule R version 17," or omits version information altogether). Breaks A2.40 directly and A4.06 transitively.

Vendor-managed-version-compatibility. A vendor system handles compatibility outside the substrate's authority architecture; version translation occurs in vendor middleware, and the substrate sees only post-translation state. Breaks A1.01 and the AI-as-substrate-mediator boundary the source paper draws at §4.1.

LLM-mediated-version-translation. An LLM is delegated authority to translate historical state to current format without an explicit rule per A2.04. This is an A3.13 instantiation — the LLM operating outside its mediator role, deciding substrate semantics rather than executing rules over substrate content. Breaks A1.01 and A2.04.

Compliance-framework-imposes-version-migration. A compliance framework or regulatory tool external to the substrate forces historical state to be migrated. Breaks A1.01 (migration occurs under authority that is not the substrate's preserved human authority) and A1.07 (historical state is altered by an actor outside the deployment's governance).

Current-rule-applied-to-historical-state-without-translation. A cell applies the current rule to historical state without verifying whether the rule's compatibility specification covers the historical version. Subtler — no explicit rewrite, no vendor middleware — but breaks A2.04 (the rule's compatibility specification, if present, was bypassed) and A4.06 (replay will not reproduce the original outcome).

These eight violations exhaust the canonical failure modes the architecture rules out.

6. Operational implications

Rule design becomes a long-lifecycle discipline: rules are authored as components of a substrate whose history they will become part of, with compatibility specified at authoring time and absence of such specification treated as architectural incompleteness. Historical state is not auto-pruned; pruning, when it occurs, is an explicit human-authored event under A1.01, governed by the substrate-near-capacity treatment formalized at A6.07 and conducted as archival that preserves rule version associations rather than outright deletion. Rule revisions are themselves substrate content with their own provenance, traceable through the same A1.07 path retraceability the rest of substrate enjoys; the deployment's history of rule versions is reconstructable from substrate content alone. Replay across rule version boundaries — the reproducibility test formalized in A5.16 — is the operational test that detects whether the architectural treatment is in place; the provenance-completeness test formalized in A5.08 verifies that A2.40 field 4 records specific versions rather than version-erasing identifiers. The treatment composes with A6.06 authority distribution change and with A6.01 conflict-as-first-class, which preserves any conflicts surfacing at version transitions as substrate content rather than dissolving them in version translation.

7. Limits of the architectural treatment

The treatment applies to rule version compatibility, not to individual rule revisions; A6.02 covers individual revisions in their immediate effect, and the two compose. It does not cover schema evolution: changes to the substrate's data structure rather than to the rules operating over the substrate are the boundary case A6.15 formalizes. A rule version may change without the schema changing; a schema may change without the rules changing; either change without the other is well-defined under the appropriate boundary case. The treatment does not cover substrate near-capacity by itself; A6.07 governs archival when long deployment history grows substrate beyond capacity, with A6.14 specifying what must be preserved across the archival. The treatment does not specify rule versioning products, backward-compatibility frameworks, or long-running-system tools — many products may serve the architecture; none is privileged by it. Nor does it commit to any particular versioning representation: rule versions may be identified by integers, content hashes, named tags, timestamps, or any other addressable form, provided field 4 of provenance carries an

identifier sufficient to address the historical rule version unambiguously.

8. Operational test

A deployment instantiates the architectural treatment of the deployment-evolution rule version compatibility boundary if and only if, for every rule version revision over the deployment's lifecycle and for every piece of substrate content authored under any rule version:

1. The specific rule version under which the content was authored is recorded in A2.40 field 4.
2. The rule under which the content was authored remains substrate-resident as long as the content remains in substrate.
3. The rule active at any subsequent moment specifies how it processes content authored under prior rule versions, either directly or by delegating to versioned interpretation logic that is itself substrate-resident.
4. Replay per A4.06 of the original execution that produced the content uses the historical rule version recorded in provenance, not the current rule version, and reproduces the original outcome.
5. Any migration of historical content to a new rule format is conducted by an explicit human-authored rule per A6.02, with the migrated content created alongside (not replacing) the original.
6. Deprecation of any rule version is recorded as a substrate-resident governance event under A1.01, not as a silent removal.

A deployment that fails any of (1)–(6) may operate, may produce useful outcomes, and may be governed in some other sense, but does not implement the CKS architectural treatment of the deployment-evolution rule version compatibility boundary, and its long-lifecycle behavior is not defended by the source paper's commitments at §5, §3.3, §3.1, or §11.3.

9. Why naming this boundary as standalone matters

The deployment-evolution rule version compatibility boundary is not a corner case affecting only edge deployments. It is the load-bearing architectural treatment for the class of deployments where CKS's traceability and reproducibility commitments matter most: long-running coordination substrates, audit-required AI deployments, regulated-industry instantiations whose historical record must remain interpretable across years. Naming it as standalone is what makes the architectural treatment defensible against deployments that satisfy CKS's commitments at a single point in time but fail them across the deployment lifecycle.

A6.14 closes the major rule-evolution arc Phase A6 carries: A6.02 covers the local effect of an individual revision, A6.06 covers authority distribution change, A6.14 covers cumulative rule version compatibility across the lifecycle. A6.15 will cover schema evolution / substrate migration as its own boundary case and close Phase A6.

* * *

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Boundary Case: Deployment-Evolution Rule Version Compatibility as Standalone Architectural Treatment — Formalizing How CKS Maintains Coherence Across Long Deployment Lifecycles Where Multiple Rule Versions Coexist, Through A2.40 Field 4 Specific Rule Version Recording and Rule-Authored Compatibility per A2.04 Without Auto-Migration of Historical State*. May 7, 2026. ORCID: 0009-0004-8065-3235.